

核心是回答一个问题：一个 Agent 从实验阶段走向生产环境前，应该如何系统地做评估？它不是单纯讲“怎么打分”，而是把 Agent 评估拆成了从观察真实行为、构造数据集、设计评估器、反复实验，到上线后持续监控和反馈闭环的完整流程。

一、核心观点

Agent 评估不是先搭一套复杂评测系统，而是先看真实 trace，搞清楚 Agent 为什么失败，再把这些失败模式转化为可重复、可自动化、可持续迭代的 eval。

也就是说，Agent eval 的起点不是“我要做一个 benchmark”，而是：

- 1. 先观察 Agent 真实执行过程；
- 2. 找到它到底在哪些地方失败；
- 3. 把失败原因分类；
- 4. 设计针对性的测试样本和 grader；
- 5. 持续把生产中的新失败反馈回数据集。

这和传统软件测试不一样。传统软件测试更多是函数输入输出是否正确；而 Agent 的行为包含模型推理、工具调用、状态变化、多轮上下文、外部环境交互，所以评估必须覆盖完整执行过程，而不只是最终回答。

二、完整流程：从实验到生产的 Agent 评估路径

这份文件可以拆成六个阶段：

阶段	解决的问题	关键词
1. 评估前准备	你是否真的理解 Agent 的失败模式？	trace、人工审查、成功标准、失败归因
2. 选择评估层级	你到底要评估 Agent 的哪一层行为？	run、trace、thread
3. 构造数据集	用什么任务来评估？	正例、反例、reference solution
4. 设计 grader	谁来判定 Agent 做得好不好？	code-based、LLM-as-judge、human
5. 运行与迭代	如何避免评估结果失真？	多次试验、隔离环境、效率指标
6. 生产就绪	如何把 eval 接入真实上线流程？	CI/CD、online eval、用户反馈、闭环

这其实就是一个 **Agent 质量管理系统**。

通俗地说：

不是“我测一下 Agent 厉不厉害”，而是“我建立一套机制，让 Agent 每次变更后都知道有没有变好==、有没有退化、为什么失败、下一步该改哪里”。

三、阶段一：在做 eval 之前，先看 20-50 条真实 trace

文件强调，做任何评估基础设施之前，应该先人工审查 20-50 条真实 Agent traces。trace 指的是 Agent 从接收任务到完成任务的完整执行轨迹，包括模型思考、工具调用、输入输出、中间状态和最终结果。

1、什么是 Agent trace？

一个 Agent trace 通常记录一次完整任务从开始到结束的过程，包括：

内容	例子
用户输入	“帮我分析本周港股市场异动”
Agent 的中间决策	是否需要查数据、是否需要调用工具
LLM 调用	prompt、model、输入输出、token、耗时
工具调用	调用了哪个 tool、参数是什么、返回了什么
中间状态	生成了哪些 JSON、CSV、docx、数据库记录
最终输出	Agent 给用户的最终回答
错误信息	timeout、schema error、API error、empty output
元数据	request_id、版本、时间、环境、prompt version

为什么这一步重要？

因为 Agent 的失败常常不是表面看到的“回答错了”，而可能是：

- prompt 没讲清楚；
- 工具设计太容易误用；
- 工具返回数据有问题；
- 模型能力不足；
- 外部 API timeout；
- 缓存过期；
- 数据管道有 bug；

- grader 本身误判。

文件举了一个很典型的例子：有团队发现一个 extraction bug 修复后，benchmark 从 50% 提升到 73%。这说明看似是 Agent 推理失败，其实可能是基础设施或数据管道问题。

通俗理解

这就像你管理一个新员工。你不能只看他最后交上来的报告好不好，而要先看：

- 他有没有理解任务；
- 有没有找错资料；
- 有没有用错工具；
- 有没有中途卡住；
- 有没有把错误信息当成事实；
- 有没有因为系统权限问题没完成。

Agent trace 就是“员工做事过程的监控录像”。

2. 一开始怎么获得 trace？有三种方式

方式一：用现成观测平台，比如 LangSmith

项目已用 LangChain/LangGraph 封装 + 添加 API key：

```
LANGCHAIN_TRACING_V2=true  
LANGCHAIN_API_KEY=你的_langsmith_key  
LANGCHAIN_PROJECT=market-insight-agent-dev
```

每次 Agent run 都会**自动记录**：

- 每次 LLM 调用；
- 每次 tool call；
- 每个 node 的输入输出；
- trace tree；
- latency；
- token usage；
- error；
- final response。

方式二：自己写轻量级 trace logger

如果你的 Agent 是自己写的，比如原生 ReAct loop、FastAPI backend、OpenAI function calling，不一定非要上 LangSmith。

你可以先做一个最小 trace 结构，把每次运行存成 JSONL 或 JSON 文件。

一开始不用追求完美，你只要回答这些问题就够了：

Agent 接了什么任务？ `task_id / user_input`

它调用了哪些 LLM？ `model`

它调用了哪些工具？ `tool_name`

每一步输入输出是什么？ `input / output / arguments / result`

哪一步失败了？ `status / error`

最终结果是什么？ `final_output / content`

例如：

```
{
  "trace_id": "20260527_001",
  "task_id": "market_anomaly_001",
  "user_input": "解释本周恒生科技指数异动",
  "steps": [
    {
      "step": 1,
      "type": "llm_call",
      "model": "deepseek-chat",
      "input": "...",
      "output": "...",
      "latency_ms": 1800
    },
    {
      "step": 2,
      "type": "tool_call",
      "tool_name": "search_market_news",
      "arguments": {
        "query": "恒生科技指数 本周 异动",
        "recency_days": 7
      },
      "result": "...",
      "latency_ms": 900
    },
    {
      "step": 3,
      "type": "final_output",
      "content": "本周恒生科技指数上涨..."
    }
  ]
}
```

```
"status": "success",  
"created_at": "2026-05-27T20:10:00"  
}
```

方式三：在本地开发时手动保存“运行快照”

如果你还在 very early MVP 阶段，甚至可以更简单：

每次跑 Agent，都保存一个文件夹：

```
runs/  
  2026-05-27_001/  
    input.json  
    prompt.txt  
    llm_response_1.json  
    tool_call_1.json  
    tool_result_1.json  
    final_output.md  
    error.log  
    metadata.json
```

这其实就是一个本地 trace。

优点是直观，适合边做项目、边理解 Agent 工程的阶段。

3. 如果是 ReAct Agent，trace 应该怎么记？

ReAct Agent 的核心循环是：

```
Thought → Action → Observation → Thought → Action → Observation → Final Answer
```

所以 trace 也可以直接按这个结构记：

```
{  
  "trace_id": "react_001",  
  "user_input": "解释本周人民币汇率异动",  
  "steps": [  
    {  
      "type": "thought",  
      "content": "需要先查看人民币汇率本周变化和相关新闻。"  
    }  
  ]  
}
```

```

    },
    {
      "type": "action",
      "tool_name": "get_market_data",
      "arguments": {
        "asset": "USDCNY",
        "period": "1w"
      }
    },
    {
      "type": "observation",
      "content": "USDCNY 本周上涨 0.8%。"
    },
    {
      "type": "action",
      "tool_name": "search_market_news",
      "arguments": {
        "query": "人民币 汇率 本周 贬值 原因",
        "recency_days": 7
      }
    },
    {
      "type": "observation",
      "content": "相关新闻包括美元指数走强、中美利差变化..."
    },
    {
      "type": "final_answer",
      "content": "本周人民币汇率变化主要受..."
    }
  ]
}

```

这里的重点不是记录模型“私密思维链”，而是记录**可观测的决策步骤和工具交互**。在生产环境里，一般不建议暴露完整 chain-of-thought，但可以记录 summary-style reasoning、tool decisions、inputs/outputs 和状态变化。

4. 一开始不要记录太多，重点记录“能帮助归因”的信息

你不是为了记日志而记日志，而是为了之后能回答：
每次失败后，我能通过 **trace** 看出失败发生在哪一步。

问题	trace 里需要有什么
Agent 是理解错任务了吗？	user_input、prompt、LLM first response
是工具选错了吗？	tool_name、tool_args
是工具返回错了吗？	tool_result、status、error
是模型编造了吗？	final_output、source evidence
是格式失败了吗？	raw output、schema validation error
是超时了吗？	latency_ms、timeout error
是状态没改对吗？	artifacts、DB rows、file paths

5. 最早期 Agent traces 可以来自哪里？

你不一定非要等真实用户。早期 traces 可以来自这几类：

1) 你自己 dogfooding

每天自己给 Agent 喂真实任务。

比如：

- 让 Market Insight Agent 分析一个市场异动；
- 让 EPUB Agent 转换一篇文章；
- 让 coding agent 修改一个小 bug；
- 让 research agent 查一个金融主题。

每次运行都保存 trace。

2) 手写 20 个典型任务

比如 Market Insight Agent：

1. 解释港股大涨
2. 解释美元指数异动
3. 解释中债收益率下行
4. 解释黄金上涨
5. 解释A股成交额放大
6. 解释北向资金流出
- ...

每条任务跑一次或多次，生成初始 trace 集合。

3) 从历史失败里补 trace

即使以前没记录完整 trace，也可以从日志、输出文件、报错信息里重建部分 trace。

比如：

- empty output;
- finish_reason=length;
- max_tokens 不够;
- PowerShell 编码错误;
- 401 Unauthorized;
- JSON schema validation failed。

这些都可以转成 eval case。

4) 从单元测试升级成 trace

你原来可能只有 test case。现在可以让每个 test case 跑 Agent，并保存完整 trace。

6. 可以按这个顺序落地

第一步：本地 JSON trace

目标：先把每次 Agent run 保存下来。

目录结构：

```
runs/  
  trace_001.json  
  trace_002.json  
  trace_003.json
```

第二步：trace review 表格

把 20-50 条 trace 人工看一遍，做一个表：

trace_id	任务	成功/失败	失败步骤	失败类型	修复建议
001	解释港股异动	失败	tool_call	搜索 query 太泛	改 search query 生成规则
002	生成周报	失败	final_output	JSON 不合法	加 schema validator
003	翻译文章	失败	llm_call	max_tokens 截断	增加分块策略

这一步就是 checklist 里说的 error analysis。

第三步：把典型失败转成 eval

比如你发现 20 条 trace 里有 5 条是 JSON 不合法，那就加一个 code-based evaluator：

```
def eval_json_schema(output):
    try:
        validate_json_schema(output)
        return True
    except Exception:
        return False
```

如果发现 5 条是“没有证据就强行归因”，那就加一个 LLM-as-judge 或规则检查：

```
检查最终解释是否把 possible driver 写成 confirmed cause。
```

7. 对你的项目，最小 trace 设计可以这样做

以你的 Market Insight Agent 为例，建议每次运行保存：

```
runs/
  2026-05-27_001/
    input.json
    data_context.json
    anomaly_candidates.csv
    agent_prompt.txt
    llm_raw_response.json
    tool_calls.json
    search_results.json
    report_insights.json
```

```
final_report.docx
trace.json
error.log
```

其中 `trace.json` 是索引文件，记录每一步对应哪个 artifact。

例如：

```
{
  "trace_id": "20260527_001",
  "task": "weekly_market_report",
  "input_files": [
    "weekly_metrics.csv",
    "anomaly_candidates.csv"
  ],
  "steps": [
    {
      "type": "data_load",
      "artifact": "data_context.json",
      "status": "success"
    },
    {
      "type": "llm_call",
      "artifact": "llm_raw_response.json",
      "status": "success"
    },
    {
      "type": "tool_call",
      "tool": "search_market_news",
      "artifact": "search_results.json",
      "status": "success"
    },
    {
      "type": "schema_validation",
      "artifact": "report_insights.json",
      "status": "success"
    },
    {
      "type": "render_report",
      "artifact": "final_report.docx",
      "status": "success"
    }
  ]
}
```

这对你后面做 eval 非常有用，因为你可以直接检查：

- 数据是否正确进入 Agent；
 - LLM 是否输出合规 JSON；
 - search 工具是否返回有效证据；
 - report_insights.json 是否通过 schema；
 - Word 是否成功渲染。
-

四、阶段二：定义清晰的成功标准

文件特别强调：每一个 eval task 都必须有明确、无歧义的成功标准。

比如：

模糊标准：

“总结这个文档。”

这个很难评估。什么叫总结得好？不同人标准可能不同。

清晰标准：

“从会议记录中提取 3 个主要行动项，每个行动项少于 20 个词，如果提到负责人则必须包含负责人。”

这个就可评估，因为它有明确约束：

- 是否有 3 个；
- 是否是行动项；
- 是否少于 20 个词；
- 是否包含负责人。

通俗理解

Agent eval 最怕的不是 Agent 答错，而是 **你自己也说不清什么叫对**。

如果两个专家对 pass/fail 无法达成一致，那不是 Agent 的问题，而是任务定义还不够清楚。

五、阶段三：区分 capability eval 和 regression eval

文件提出一个非常关键的分类：

1. Capability eval：能力评估

它回答的是：

Agent 现在能做到什么？

这类 eval 可以很难，一开始通过率低也没关系。它的作用是帮助你看到能力边界，推动 Agent 进步。

比如：

- Agent 能不能完成复杂代码修改？
- 能不能多工具协同查资料？
- 能不能在多轮对话中保持上下文？
- 能不能判断什么时候需要追问用户？

Capability eval 的典型流程

可以按这个流程做：

定义目标能力

- 设计偏难任务
- 准备 **reference solution** 或判断标准
- 跑 **Agent** 多次
- 用 **code-based** / **LLM-as-judge** / **human grader** 评分
- 分析失败类型
- 改 **prompt** / **tool** / **model** / **workflow**
- 再跑实验

举个例子：

目标能力：

Agent 能否对市场异动做“证据约束型解释”。

测试任务：

输入 **anomaly_candidates.csv** 中 5 个异常，让 **Agent** 生成解释卡片。

成功标准：

1. 必须引用数据事实；
2. 必须说明异常幅度；
3. 外部新闻只能作为 **possible driver**；
4. 不得写成确定因果；
5. 输出必须符合 **report_insights.json schema**。

评估方式：

schema 用 code-based evaluator;
事实引用用规则检查;
因果措辞用 LLM-as-judge;
最终样本人工复核。

2. Regression eval: 回归评估

它回答的是:

Agent 以前能做的事情, 现在是否还稳定?

这类 eval 应该接近 100% 通过率, 用来防止系统改动后出现退化。

比如:

- 上个版本可以正确生成 Word 报告, 新版本不能了;
- 之前能正确调用 search 工具, 现在 prompt 改了以后不调用了;
- 之前能处理中文文件名, 现在 encoding 改动后报错。

如果你只做 capability eval, 可能不断追新能力, 却把老功能搞坏; 如果只做 regression eval, 系统会很稳, 但不会进步。

Regression eval 应该接入 CI/CD

Regression eval 最好不是人工偶尔跑, 而是每次关键变更自动跑。

触发条件可以是:

- 修改 prompt;
- 修改 tool schema;
- 修改模型;
- 修改数据处理逻辑;
- 修改 report renderer;
- 修改 Agent loop;
- 上线前手动触发。

典型流程:

```
git push / prompt update
→ 运行单元测试
→ 运行 regression eval
→ 检查质量门禁
→ 通过才允许 merge / deploy
→ 失败则输出失败 trace
```

质量门禁可以设置成：

```
核心 regression eval 通过率必须 = 100%
非核心 regression eval 通过率必须 >= 95%
任何 schema error / data loss / safety violation 直接阻断上线
```

Regression eval 要优先用确定性 grader

因为 regression eval 是上线保护，所以要尽量减少主观波动。

优先级是：

```
code-based evaluator > rule-based evaluator > calibrated LLM-as-judge > human
review
```

例如：

```
def eval_report_schema(report_json):
    required_fields = ["asset_name", "metric_name", "fact", "magnitude",
"possible_drivers"]
    for card in report_json["cards"]:
        for field in required_fields:
            if field not in card:
                return False
    return True
```

这种比让 LLM 判断“报告质量是否好”更稳定。

Regression eval 的数据集不宜无限膨胀

Regression eval 不是越多越好。它应该保留最能代表生产行为的核心 case。

可以分成三类：

P0：必须通过

一失败就不能上线。

例如：

- JSON schema 不合法；
- 文件生成失败；
- 工具参数非法；
- 数据字段丢失；
- 输出为空；
- 把无证据推断写成确定事实。

P1：应该通过

失败后需要修复，但不一定阻断所有发布。

例如：

- 解释不够完整；
- 口播风格不自然；
- 某些边界 case 表达不够好。

P2：观察项

用于长期监控，不作为硬门禁。

例如：

- token 使用量；
- step count；
- latency；
- 搜索调用次数；
- 输出长度。

六、阶段四：选择评估层级：run、trace、thread

文件把 Agent eval 分成三个层级：

层级	对应对象	评估什么	适用场景
Single-step / Run-level	单一步骤	工具是否选对、API call 是否合法	调试具体工具调用
Full-turn / Trace-level	一轮完整任务	最终回答、执行路径、状态变化	最推荐的起点
Multi-turn / Thread-level	多轮对话	上下文保持、长期任务推进	高阶 Agent

文件建议大多数团队从 **trace-level full-turn eval** 开始，因为它信息量最大。

1. Run-level eval

评估单一步骤，例如：

- Agent 是否调用了正确工具；
- tool call 参数是否合法；
- 是否生成了符合 schema 的 JSON；
- 是否选择了正确 API。

它适合调试，但缺点是容易被架构变动影响。如果工具定义经常变化，run-level eval 会比较脆弱。

2. Trace-level eval

这是最实用的层级。它评估一轮完整任务，包括三部分：

1. **Final response**：最终回答是否正确、有用；
2. **Trajectory**：执行路径是否合理；
3. **State changes**：外部状态是否真的被正确改变。

第三点非常重要。文件强调，Agent 不只是“说了什么”，还要看它“做成了什么”。

比如日程 Agent 说：

“会议已经安排好了。”

这还不够。你要检查：

- 日历里是否真的有这个会议；
- 时间是否正确；
- 参会人是否正确；
- 描述是否正确；
- 是否发出了邀请。

3. Thread-level eval

这是多轮对话级别评估，最难实现。它评估 Agent 能不能在多轮任务中记住上下文、处理用户反复修正、逐步完成复杂目标。

文件建议使用 **N-1 testing**：从真实生产对话中截取前 N-1 轮，只让 Agent 生成最后一轮。这样可以避免完全模拟多轮对话时错误逐轮累积。

通俗理解

这三个层级可以类比成：

- run-level：看员工某一步操作是否正确；
- trace-level：看员工是否完整完成了一件事；
- thread-level：看员工是否能长期跟进一个项目。

七、阶段五：构造高质量数据集

文件强调，数据集不是越大越好，而是要高质量、可验证、贴近真实生产行为。核心原则包括：

1. 每个任务都要无歧义，并有 reference solution

也就是你必须知道这个任务是可完成的，而且有一个参考答案或参考状态。

例如：

模糊任务：

“帮我找去纽约的好航班。”

清晰任务：

“找 SFO 到 JFK 的往返航班，12 月 15-17 日出发，12 月 22 日返回，经济舱，价格低于 400 美元。”

后者才适合作为 eval task。

2. 同时测试正例和反例

不能只测试 Agent “该做什么”，也要测试 Agent “不该做什么”。

比如搜索 Agent：

- 正例：用户问近期新闻时，应该搜索；
- 反例：用户问常识性问题时，不应该乱搜索；
- 正例：信息不足时，应该追问；
- 反例：信息已经足够时，不应该重复追问。

否则你会训练出一个“什么都搜索”的 Agent。

3. 数据集结构要匹配评估层级

不同 eval 层级需要不同数据结构：

- run-level：需要参考 tool call 或参考决策；
- trace-level：需要预期最终结果和状态变化；
- thread-level：需要多轮对话序列和上下文保持要求。

4. 根据 Agent 类型定制数据集

文件区分了几类 Agent：

- **Coding Agent**：要有确定性测试，比如 unit tests；
- **Conversational Agent**：要评估任务完成度、表达清晰度、共情能力；
- **Research Agent**：要评估 groundedness，即结论是否有来源支持，以及 coverage，即关键事实是否覆盖。

5. 冷启动时先手写 20-50 个高质量样本

文件明确说，20-50 个经过人工认真审查的样本，往往比几百个未经验证的合成样本更有价值。

这点很重要，因为 eval 数据的质量决定了你是否会优化错方向。

八、阶段六：设计 grader，不要迷信一个万能评分器

文件把 grader 设计讲得非常实用：不同维度要用不同 grader，不要试图用一个“智能评分器”解决所有问题。

Grader 类型

Grader 类型	适合评估什么	风险
Code-based evaluator	格式、工具调用、状态变化、执行结果	可能误杀“格式不同但正确”的答案
LLM-as-judge	开放式质量、主观判断、rubric 评分	需要人工校准
Human evaluator	模糊任务、边界案例、主观质量	成本高、速度慢
Pairwise evaluator	比较两个版本哪个更好	不适合绝对质量判断

1. 客观任务优先用 code-based evaluator

如果答案客观可验证，就不要让 LLM 来主观判断。

比如：

- JSON schema 是否合法；
- API 参数是否正确；
- 文件是否生成；
- 单元测试是否通过；
- 数据库记录是否更新；
- 日历事件是否存在。

这些都应该用代码判断。

2. 主观任务才用 LLM-as-judge

例如：

- 回答是否有帮助；
- 解释是否清楚；
- 是否符合用户语气；
- 是否覆盖关键事实；
- 是否有合理推理。

但 LLM-as-judge 必须校准。文件建议从 20+ 人工标注样本开始，逐步增加到约 100 个，以提升生产级信心。

3. 区分 guardrails 和 evaluators

这是文件里一个很关键的工程区分：

类型	什么时候运行	目的
Guardrails	执行中，用户看到结果前	阻止危险、错误格式、敏感输出
Evaluators	生成后，异步运行	评估质量、发现退化

通俗地说：

- guardrail 是“刹车系统”；
- evaluator 是“质检系统”。

不要把两者混在一起。

4. 优先使用 binary pass/fail，而不是 1-5 分

文件建议大多数情况下用二元判断：通过 / 不通过。

因为 1-5 分很容易产生主观差异，比如 3 分和 4 分到底差在哪里，不同人可能标准不同。**二元判断会迫使你把成功标准定义清楚。**

复杂任务可以拆成多个 binary checks：

- 是否完成任务；
- 是否引用来源；
- 是否调用正确工具；
- 是否更新状态；
- 是否没有编造；
- 是否格式正确。

九、阶段七：评估时看结果，不要死盯路径

文件引用了 Anthropic 的观点：**不要评价 Agent 走了哪条路，而要评价它最终产出了什么。**

比如会议安排任务，不应该强制要求：

| 必须先 call check_availability，再 call create_event。

更好的评估是：

| 最终会议是否被正确安排？

因为 Agent 可能找到更高效路径。如果你强行要求固定工具调用顺序，就会误判一些合理解法。

但文件也补充，虽然 correctness 不应该死盯路径，但可以记录效率指标，比如：

- step count；
- tool calls；
- latency；
- token usage；
- cost per task；
- observed steps / ideal steps；
- observed latency / ideal latency。

也就是说：

路径不一定决定对错，但路径可以衡量效率。

通俗理解：

员工完成任务的方式可以不同，但如果别人 3 分钟完成，他绕了 30 分钟，你也要知道这个问题。

十、阶段八：运行 eval 时要考虑非确定性

Agent 输出具有非确定性，同一个任务多跑几次结果可能不一样。因此文件建议：

- 每个 task 可以跑多次 trial；
- 不要只看单次结果；
- 成本允许时计算置信区间；
- 可使用 $\text{pass}@k$ 或 pass^k 。

pass@k

只要 k 次尝试中有一次成功，就算成功。

适合场景：用户可以重试，或者系统允许多次尝试。

pass^k

k 次都必须成功，才算成功。

适合场景：金融、合规、生产自动化等高可靠性任务。

对你的 Agent 项目来说，如果是金融市场周报、数据分析、自动化报告，很多场景更接近 pass^k ，因为你希望它稳定，而不是偶尔成功。

十一、阶段九：每次 trial 必须隔离环境

文件强调，每次评估都要在干净、隔离的环境里运行，否则结果会被污染。

例如：

- Coding Agent：每次用新的 container 或 VM；
- API Agent：使用 staging 环境或 mock service；
- Database Agent：每次恢复数据库快照；
- 文件操作 Agent：每次准备新的工作目录。

否则 trial 2 可能读到了 trial 1 留下的文件，导致你误以为 Agent 成功了。

这对你做 Agent 项目很关键。比如你的 Market Insight Agent 或 Economist EPUB 项目，如果 Agent 生成了中间 CSV、JSON、docx，每次 eval 都应该清空或隔离工作目录，否则结果不可信。

十二、阶段十：生产就绪：把 eval 接入 CI/CD 和线上反馈

CI = Continuous Integration 持续集成

CD = Continuous Deployment 持续部署

文件最后讲的是从实验到生产的关键迁移：eval 不能停留在 notebook 或本地脚本里，而要进入工程流程。

典型 CI/CD 流程是：

1. 代码或 prompt 变更触发 pipeline；
2. 离线 eval 运行单元测试、集成测试、数据集评估；
3. 通过后部署到 preview 环境；
4. preview 环境运行 online eval；
5. 所有质量门禁通过后才上线；
6. 失败 trace 进入 annotation queue(「需要人工复核 / 打标签的失败 Trace 的待办队列」)；
7. 团队分析失败并更新数据集和 grader。

生产环境还要做三件事

第一，收集用户反馈。

自动 eval 只能发现已知失败模式，用户会暴露你没想到的问题。

第二，定期人工探索生产 trace。

不要只依赖自动 pass/fail。很多新问题一开始不会被现有 grader 捕捉。

第三，版本化 prompt 和 tool definitions。

如果 prompt、工具定义、模型版本没有版本管理，你就无法回答：

- 是哪个 prompt 改动导致退化？
 - 换模型后哪些能力提升？
 - 哪个工具定义变更导致 tool call 错误增加？
-

十三、最核心的飞轮：Production Trace → Annotation → Dataset → Experiment → Online Eval

文件第 25 页的图展示了一个闭环：

1. **Production Traces**：生产环境暴露真实 case；
2. **Annotation Queues**：人工审查和标注 trace；
3. **Datasets**：把有代表性的 case 加入数据集；
4. **Experiments**：用实验验证改动是否改善；
5. **Online Evals**：上线后持续监控；
6. 然后重复。

这个飞轮是整篇文章的灵魂。

通俗地说：

好 Agent 不是一次性调出来的，而是在真实使用中不断发现失败、沉淀样本、修正系统、重新评估，逐步“训”出来的。

十四、用一句话解释每个关键概念

Trace

Agent 完成一次任务的完整过程记录。

就像“做事录像”。

Run

Trace 里的某一步。

就像“某一次工具调用”或“某一次模型响应”。

Thread

多轮对话的完整上下文。

就像“一个长期项目的全部沟通记录”。

Dataset

用来反复测试 Agent 的任务集合。

就像“考试题库”。

Reference solution

每道题的参考答案或参考状态。

就像“标准答案”。

Grader

判断 Agent 是否做对的机制。

就像“阅卷老师”。

Guardrail

用户看到结果前的安全和格式拦截。

就像“上线前的安全阀”。

Offline eval

上线前在固定数据集上测试。

就像“模拟考试”。

Online eval

上线后在真实流量中监控。

就像“真实工作表现抽检”。

Regression suite

防止老能力退化的测试集。

就像“基本功检查”。

十五、对你当前做 AI Agent 项目的启发

结合你之前做的 **Market Insight Agent** 和 **Economist EPUB → 中文口播稿系统**，这份文件对你最有价值的地方是：它提供了一套从 MVP 到生产化的 Agent 质量管理方法。

你可以这样迁移：

1. 对 Market Insight Agent

你可以设计三层 eval:

Run-level:

- 是否正确读取 weekly_metrics.csv;
- 是否正确识别 anomaly_candidates.csv;
- 是否调用 search_market_news;
- tool call 参数是否包含 asset、metric、recency_days。

Trace-level:

- 是否生成四段式文本卡片;
- 是否区分事实、幅度、影响、可能驱动;
- 是否避免把外部新闻线索写成确定因果;
- 是否在证据不足时保守表达。

State-change / artifact-level:

- 是否生成 report_insights.json;
- JSON schema 是否通过;
- Word 报告是否成功渲染;
- 每个 anomaly 是否都有对应解释卡片。

2. 对 Economist EPUB 项目

可以设计 eval:

客观 code-based eval:

- EPUB 是否成功解析;
- 目录、广告、导航是否被过滤;
- 长文是否分块;
- docx 是否生成;
- 输出是否非空;
- UTF-8 编码是否正常;
- max_tokens 截断是否被识别。

主观 LLM-as-judge eval:

- 中文口播稿是否自然;
- 是否忠实原文;
- 是否适合播客语气;

- 是否有明显遗漏；
- 是否有编造内容。

Regression eval:

- 选取 20 篇历史文章作为固定回归集；
 - 每次改 prompt、模型、分块策略、清洗规则后都跑一遍；
 - 防止“翻译变好但结构坏了”或“速度变快但漏译增加”。
-

十六、这份文件真正想让你形成的工程意识

1. 先观察，再自动化

不要一上来就写 eval 框架。先看真实 trace，理解失败模式。

2. 先定义成功，再评估成功

如果成功标准不清楚，评估系统越复杂越危险。

3. 先小样本高质量，再大规模扩展

20-50 个高质量人工审查样本，胜过几百个未经验证的合成样本。

4. 先评估完整任务，再细拆局部步骤

大多数 Agent 应先从 trace-level eval 开始。

5. 先用确定性判断，再用 LLM 判断

能用代码判断的，不要交给 LLM-as-judge。

6. 评估不是一次性动作，而是生产飞轮

生产失败必须回流到 dataset、error analysis 和 eval 改进中。

十七、最终总结

这份文件的核心不是教你“怎么给 Agent 打分”，而是教你建立一个 **Agent 生产质量闭环**。

它的逻辑是：

真实 **trace** 暴露失败 →
人工分析失败原因 →
构造高质量 **eval** 数据集 →
设计针对性 **grader** →
离线实验验证改动 →
接入 **CI/CD** 防止回归 →
上线后继续 **online eval** 和用户反馈 →
再回流到数据集。

不是 demo 能不能跑，而是 Agent 能不能被可靠地评估、迭代、上线和持续改进。